

VOLUMEN I

Datos, Seguridad y Extensibilidad

Modelo de datos, seguridad y privacidad, MCP y skills
— compilado en un único documento.

- | | | |
|----|------------------------|--|
| 07 | MODELO DE DATOS | Entidades, relaciones, persistencia, índices y migraciones |
| 08 | SEGURIDAD Y PRIVACIDAD | Amenazas, criptografía, modos de privacidad, gestión de incidentes |
| 09 | MCP Y SKILLS | Tools, scopes, agentes, autonomía y marketplace futuro |

Tabla de Contenidos

Este volumen compila 3 documentos técnicos de VNBOT: Modelo de Datos, Seguridad y Privacidad, y MCP y Skills. Cada documento preserva su numeración original para facilitar la referencia cruzada con el resto del proyecto.

1. Propósito	7
2. Principios del modelo	7
3. Capas de almacenamiento	8
4. Convenciones	8
4.1. IDs	8
4.2. Timestamps estándar	8
5. Entidades principales de identidad	8
5.1. User	8
5.2. Workspace	8
5.3. Roles y autorización	9
6. Conversation y Message	9
7. MemoryNode — entidad principal de memoria	9
7.1. Tipos de nodos	9
7.2. Authority y sensitivity	9
8. MemoryEdge — relaciones entre nodos	10
9. EmbeddingIndex — datos derivados	10
10. Reminder y ReminderOccurrence	10

11. Agent, Skill y ToolPermission	10
11.1. Agent	10
11.2. Skill con manifest YAML	10
11.3. ToolPermission — deny by default	10
12. Operation, ExecutionLog y Job	11
13. Relaciones principales entre entidades	11
14. Estados y transiciones	11
15. Índices recomendados y campos cifrados	11
15.1. Índices de base de datos	11
15.2. Campos potencialmente cifrados	11
16. Criterios de aceptación del modelo	12
17. Conclusión	12
1. Propósito	13
2. Principios de seguridad	14
3. Modelo de amenaza	14
3.1. Activos protegidos	14
3.2. Actores de amenaza	15
4. Modos de privacidad	16
5. Zero-knowledge: definición precisa	16
6. Criptografía	17

- 6.1. Datos en reposo 17
- 6.2. Derivación de claves 17
- 6.3. Claves por capas 17
- 7. Gestión de identidad 17**
 - 7.1. Contraseñas y sesiones 17
- 8. Autorización y aislamiento 17**
 - 8.1. Capas de autorización 17
 - 8.2. Workspace isolation 18
- 9. Seguridad del frontend 18**
- 10. Seguridad del backend 18**
- 11. Seguridad de LLM 19**
 - 11.1. Prompt injection y separación de instrucciones 19
 - 11.2. Tool calling — el LLM propone, no ejecuta 19
- 12. Seguridad de MCP 19**
 - 12.1. Herramientas peligrosas que requieren confirmación fuerte 19
 - 12.2. Aislamiento 20
- 13. Logs, auditoría y telemetría 20**
 - 13.1. Logs operativos permitidos 20
 - 13.2. No registrar por defecto 20
- 14. Backups y recuperación 20**
- 15. Gestión de incidentes 21**

15.1. Clasificación	21
15.2. Flujo de respuesta	21
16. Testing de seguridad	21
16.1. Automatizado en CI (obligatorio desde día uno)	21
16.2. Antes de cada release	21
16.3. Antes de v1.0	21
17. Criterios de aceptación de seguridad	22
18. Conclusión	23
1. Propósito	24
2. Principios	24
3. Arquitectura MCP de VNBOT	25
3.1. MCP interno (13 tools)	25
3.2. MCP externo	25
4. Registro de servidores MCP	25
4.1. Flujo de registro	25
4.2. Registro local (stdio)	25
4.3. Registro remoto (Streamable HTTP)	25
5. Scopes y permisos	26
5.1. Scopes iniciales (15 scopes)	26
5.2. Niveles de permiso	26
5.3. Matriz de permisos comprensible	26

6. Clasificación de riesgo de herramientas	26
7. Confirmaciones	26
7.1. Confirmación simple (riesgo bajo/medio)	26
7.2. Confirmación fuerte (riesgo alto)	27
7.3. Confirmación caducada	27
8. Skills	27
8.1. Estructura de una skill	27
8.2. Manifest YAML	27
8.3. Skills iniciales (19 skills)	27
9. Ciclo de vida de una skill	27
9.1. Actualización de skills	27
10. Agentes	28
10.1. Configuración de un agente	28
10.2. Agentes iniciales (7 agentes)	28
11. Niveles de autonomía (0-4)	28
12. Memoria accesible por agente	28
12.1. Scopes de memoria	28
12.2. Reglas de memoria por agente	29
12.3. Context assembly	29
13. Presupuesto y límites	29
13.1. Límites por agente	29
13.2. Exceso de presupuesto	29

14. Tool execution — preflight, ejecución y post	29
14.1. Preflight (antes de ejecutar)	29
14.2. Ejecución	30
14.3. Post-execution	30
15. Contract testing para herramientas MCP	30
15.1. Métricas de tools	30
16. Criterios de aceptación	31
16.1. MCP está correctamente implementado cuando (13 criterios)	31
16.2. Una skill está lista cuando	31
16.3. Un agente está listo cuando	31
17. Conclusión	32

DOC 07 • MODELO DE DATOS

Entidades, relaciones, identificadores, estados, reglas de integridad y formatos de persistencia.

1. Propósito

Este documento define las entidades, relaciones, identificadores, estados, reglas de integridad y formatos de persistencia de VNBOT. El modelo debe soportar instalación local con SQLite, PWA con IndexedDB, servidor con PostgreSQL, memoria de nodos y relaciones, recordatorios puntuales y recurrentes, agentes y skills, integraciones MCP, jobs asíncronos, archivos y audio, auditoría, exportación/importación y sincronización futura entre dispositivos.

El modelo no debe asumir que todo el contenido estará en texto plano. Algunos campos se almacenarán cifrados, algunos serán metadatos operativos y otros serán índices derivados.

2. Principios del modelo

Principio	Descripción
Aislamiento por workspace	Toda entidad relacionada con datos del usuario debe tener un workspace_id directo o heredado de una relación autorizada.
Procedencia obligatoria	Toda memoria o relación generada por IA debe indicar cómo fue creada: explicit_user_input, user_confirmed, conversation_extraction, audio_transcription, image_ocr, llm_inference, external_integration, imported_data, system_generated.
Confianza ≠ autoridad	confidence (qué tan seguro está el sistema) y authority (si el usuario confirmó) son diferentes. Una inferencia con confianza 0.98 no equivale a una preferencia confirmada.
Borrado gradual	Los datos pasan por: active → deleted/soft_deleted → purged. La purga definitiva debe incluir índices, vectores, archivos, cache y referencias derivadas.
Fechas UTC + zona explícita	Los timestamps técnicos se guardan en UTC. Las operaciones de usuario también deben conservar la zona horaria de interpretación.
Identificadores opacos	No usar emails, nombres o índices secuenciales como IDs públicos. Usar UUID/ULID o identificadores equivalentes.

3. Capas de almacenamiento

Capa	Tipo	Contenido
Datos de dominio	Fuente de verdad	Usuarios, workspaces, memorias, relaciones, recordatorios, agentes, integraciones, jobs.
Datos derivados	Reconstruibles	Embeddings, índices de búsqueda, resúmenes, estadísticas, layout del grafo, cache.
Archivos	Storage externo	Audio, imágenes, documentos, backups, assets generados. La base de datos conserva referencia segura y metadata.

Adaptadores de persistencia:

```
IndexedDBAdapter → PWA<br/>SQLiteAdapter → local/desktop<br/>PostgresAdapter → server<br/>FileAdap
```

4. Convenciones

4.1. IDs

```
usr_01J...<br/>ws_01J...<br/>mem_01J...<br/>edge_01J...<br/>rem_01J...<br/>occ_01J...<br/>job_01J...
```

La parte legible ayuda a debugging, pero no debe revelar información personal.

4.2. Timestamps estándar

```
created_at · updated_at · deleted_at · expires_at
```

Todos en UTC ISO 8601 o tipo timestamp con zona en la base de datos.

5. Entidades principales de identidad

Las entidades de identidad siguen un modelo jerárquico: User → Session/Device → Workspace → WorkspaceMember.

5.1. User

```
User<br/>- id: string PK<br/>- email: string nullable, unique cuando exista<br/>- display_name: string
```

Reglas: no almacenar contraseña en texto, email puede ser nullable en modo local, no usar email como ID, la eliminación de usuario debe revocar sesiones e integraciones, las memorias pertenecen a workspaces no directamente a emails.

5.2. Workspace

Un workspace es el límite de datos, memoria y permisos. Tipos: personal, family, team, project.

```
Workspace<br/>- id, owner_user_id, name<br/>- type: personal|family|team|project<br/>- status: active|
```

Un recurso no puede cruzar workspaces sin una operación explícita. Un agente debe pertenecer a un workspace. Las integraciones deben estar vinculadas a un workspace concreto.

5.3. Roles y autorización

- **owner** — propietario del workspace, control total.
- **admin** — administración sin poder eliminar el workspace.
- **member** — uso completo de memoria y recordatorios.
- **viewer** — solo lectura.

6. Conversation y Message

```
Conversation<br/>- id, workspace_id, user_id, agent_id nullable<br/>- title nullable, status: active|a
```

Reglas: el contenido puede estar cifrado, content_preview debe ser opcional y no contener secretos, los mensajes de herramientas deben referenciar la ejecución no copiar credenciales, la retención de conversación debe ser configurable.

7. MemoryNode — entidad principal de memoria

Es la entidad principal de memoria personal. Contenido cifrado, procedencia obligatoria, confianza separada de autoridad.

```
MemoryNode<br/>- id, workspace_id, type, label<br/>- content_ciphertext, content_format<br/>- structur
```

7.1. Tipos de nodos

```
person · place · project · task · reminder · event<br/>preference · note · document · conversation · a
```

7.2. Authority y sensitivity

Campo	Valores
authority	explicit · user_confirmed · system_extracted · inferred · external_source
sensitivity	public · personal · sensitive · secret

8. MemoryEdge — relaciones entre nodos

```
MemoryEdge<br/>- id, workspace_id<br/>- source_node_id, target_node_id<br/>- relation_type, properties
```

Relaciones iniciales (13 tipos):

```
KNOWS · WORKS_ON · RELATED_TO · DEPENDS_ON<br/>REMINDER_FOR · HAPPENS_AT · PREFERS · SUPERSEDES<br/>C
```

9. EmbeddingIndex — datos derivados

Los embeddings son datos derivados y deben poder regenerarse desde la fuente de verdad.

```
EmbeddingIndex<br/>- id, workspace_id<br/>- entity_type: memory_node|message|document<br/>- entity_id,
```

No crear embeddings remotos sin consentimiento. El vector debe estar limitado al workspace. Si se borra el contenido, se borra o invalida el embedding.

10. Reminder y ReminderOccurrence

La separación entre Reminder (regla) y ReminderOccurrence (instancia) permite generar ocurrencias con anticipación controlada sin duplicar filas infinitamente.

```
Reminder<br/>- id, workspace_id, created_by_user_id, created_by_agent_id nullable<br/>- title, descrip
```

11. Agent, Skill y ToolPermission

11.1. Agent

```
Agent<br/>- id, workspace_id, name, description<br/>- avatar_id, system_prompt_ciphertext<br/>- model
```

11.2. Skill con manifest YAML

```
id: reminder.create<br/>version: 1.0.0<br/>risk_level: low<br/>required_tools:<br/>&nbsp;&nbsp;&nbsp;- remi
```

11.3. ToolPermission — deny by default

```
ToolPermission<br/>- id, workspace_id, agent_id, integration_id nullable<br/>- tool_name<br/>- permiss
```

Deny por defecto. Una skill no puede ampliar permisos por sí sola. Un agente no puede usar una herramienta sin permiso vigente. La revocación debe ser inmediata para nuevas llamadas.

12. Operation, ExecutionLog y Job

Estas entidades dan trazabilidad completa a cada acción del usuario o agente.

Operation
- id, workspace_id, user_id, agent_id nullable
- conversation_id nullable, type, st

13. Relaciones principales entre entidades

User 1 — N WorkspaceMember
Workspace 1 — N MemoryNode
Workspace 1 — N MemoryEdge
Mem

14. Estados y transiciones

Entidad	Transiciones permitidas
MemoryNode	active → superseded · active → expired · active → deleted · superseded → deleted · expired → deleted
Reminder	active → paused · active → completed · active → cancelled · paused → active · paused → cancelled
ReminderOccurrence	pending → queued · queued → sending · sending → delivered · sending → failed · failed → retrying · retrying → sending · pending → cancelled · pending → snoozed · snoozed → pending
Integration	disconnected → connecting · connecting → healthy · connecting → degraded · healthy → reauth_required · healthy → revoked · degraded → healthy

15. Índices recomendados y campos cifrados

15.1. Índices de base de datos

users(email)
workspace_members(user_id, workspace_id)
memory_nodes(workspace_id, type, status)

15.2. Campos potencialmente cifrados

- Message.content_ciphertext
- MemoryNode.content_ciphertext y structured_data_ciphertext
- MemoryEdge.properties_ciphertext
- Reminder.description_ciphertext
- Agent.system_prompt_ciphertext
- FileAsset.original_name_ciphertext
- CredentialRef.encrypted_secret
- ExecutionLog.result_summary_ciphertext

16. Criterios de aceptación del modelo

El modelo se considera preparado cuando cumple 16 criterios:

01. Se puede representar una memoria explícita y una memoria inferida.
02. Se puede conservar procedencia y editar/versionar.
03. Se puede relacionar nodos e invalidar una relación.
04. Se puede crear recurrencia sin duplicar ocurrencias.
05. Se puede registrar delivery.
06. Se puede asignar una skill a un agente y revocar una tool permission.
07. Se puede auditar una operación.
08. Se puede almacenar un audio temporal.
09. Se puede exportar e importar.
10. Se puede utilizar SQLite y PostgreSQL.
11. Se puede purgar un dato y sus derivados.
12. Se puede sincronizar con detección de conflictos futura.

17. Conclusión

El modelo de datos de VNBOT separa claramente identidad, workspaces, conversaciones, memorias, relaciones, recordatorios, ocurrencias, notificaciones, agentes, skills, integraciones, jobs, archivos, auditoría y datos derivados.

La fuente de verdad debe ser portable y comprensible; los índices, embeddings, caches y respuestas de IA deben ser derivados y reemplazables.

DOC 08 • SEGURIDAD Y PRIVACIDAD

Modelo de amenaza, criptografía, modos de privacidad, gestión de incidentes y testing de seguridad.

1. Propósito

VNBOT procesará información que puede ser íntima o sensible: recordatorios, relaciones personales, conversaciones, audios, documentos, datos de calendario, preferencias, correos y credenciales de integraciones. Este documento define cómo proteger esa información en todas las capas: Web/PWA, APK, desktop, CLI, backend, workers, base de datos, object storage, LLM Router, MCP Gateway, integraciones externas, CI/CD, backups y comunidad open source.

La seguridad no se limita a cifrar tablas. VNBOT debe cumplir 7 objetivos:

- Reducir la cantidad de datos recogidos.
- Separar datos por usuario y workspace.
- Limitar qué puede leer cada agente.
- Explicar cuándo los datos salen del dispositivo.
- Evitar acciones irreversibles sin confirmación.
- Permitir exportar, corregir y eliminar.
- Mantener trazabilidad sin registrar contenido privado innecesario.

2. Principios de seguridad

Principio	Descripción
Privacidad por defecto	Las funciones que impliquen proveedores externos, lectura de correo, envío de mensajes o acceso a archivos deben estar deshabilitadas por defecto.
Mínimo privilegio	Cada usuario, agente, skill, integración y job recibe únicamente los permisos necesarios para su operación.
Separación de secretos y contenido	Una API key, token OAuth o contraseña no son memorias normales. Deben residir en un almacén de secretos separado y no aparecer en logs, embeddings, prompts ni exportaciones normales.
Transparencia operacional	El usuario debe poder saber qué modelo se utilizó, qué herramienta se llamó, qué datos se enviaron, qué acción fue ejecutada, qué quedó pendiente y qué se guardó como memoria.
El LLM no es frontera de seguridad	Las instrucciones del modelo pueden estar equivocadas o manipuladas. La autorización se aplica fuera del prompt mediante código determinista.
Fail closed para riesgo	Si el sistema no puede verificar permiso, identidad, fecha, destinatario o integridad, debe detener la operación y pedir revisión.
Portabilidad	El usuario debe poder exportar sus datos en un formato documentado y no depender exclusivamente de un proveedor.
Eliminación verificable	Borrar un dato implica considerar fuente, versiones, índices, embeddings, cache, archivos y backups según retención.

3. Modelo de amenaza

3.1. Activos protegidos

- Bóveda de memoria, mensajes y conversaciones.
- Recordatorios y relaciones del grafo.
- Audios, imágenes y documentos.
- Calendarios, correos y contactos de terceros.
- API keys y tokens OAuth.
- Configuración de agentes y prompts privados.
- Skills instaladas, backups, logs y trazas.

3.2. Actores de amenaza

Actor	Objetivo
Atacante remoto	Explotar API, autenticación, archivos, SSRF o dependencias.
Atacante con XSS	Ejecutar JavaScript en el navegador para robar sesión, modificar acciones o acceder a datos descifrados durante una sesión activa.
Servidor comprometido	Leer base de datos, archivos, variables de entorno o procesos.
Proveedor LLM externo	Recibir contexto si el usuario lo autoriza. El riesgo depende de su política, retención, ubicación y configuración.
Servidor MCP malicioso	Devolver instrucciones manipuladoras, solicitar scopes excesivos o intentar extraer datos.
Skill maliciosa	Usar instrucciones para pedir herramientas no necesarias o provocar acciones peligrosas.
Extensión/navegador comprometido	Leer el DOM o interceptar eventos durante la sesión web.
Persona con acceso al dispositivo	Acceder a archivos locales, backups, sesiones abiertas o claves almacenadas en el sistema.
Usuario malicioso dentro de workspace	Consultar memorias, archivos o integraciones de otro miembro.

Límites explícitos — no se puede garantizar protección absoluta contra:

- Un sistema operativo completamente comprometido.
- Malware con acceso al proceso desbloqueado.
- Un usuario que entregue voluntariamente sus credenciales.
- Un administrador del servidor que controle la máquina y las claves de ejecución.
- Un proveedor externo que procese datos enviados bajo su propia política.

4. Modos de privacidad

Modo	Características	Límites
Local Estricto	Memoria en SQLite/IndexedDB local, embeddings locales, audio local, LLM local (Ollama, llama.cpp), sin sync remota por defecto, sin analytics, exportación manual cifrada.	Si el dispositivo está comprometido o desbloqueado, el atacante podría acceder a datos procesados en memoria.
Servidor Privado	API y DB en servidor elegido, acceso desde varios dispositivos, PostgreSQL, Redis y storage opcional, LLM local o proveedor configurado.	El administrador del servidor puede tener capacidad técnica para inspeccionar procesos, archivos o memoria. El cifrado en reposo no equivale automáticamente a zero-knowledge.
LLM Externo	El usuario selecciona proveedor y modelo, se envía el contexto mínimo necesario, memorias secretas pueden bloquearse para proveedores externos, se muestra el proveedor utilizado.	Antes de activar este modo se debe mostrar: proveedor, modelo y datos excluidos.

5. Zero-knowledge: definición precisa

En VNBOT, "zero-knowledge" solo debe utilizarse si el componente concreto no puede leer el plaintext, incluyendo el flujo de interpretación necesario.

Caso	¿Servidor ve plaintext?	Descripción correcta
Bóveda local + LLM local	No, fuera del dispositivo	Procesamiento local
DB con ciphertext + API cloud para analizar texto	Sí durante la solicitud	Cifrado en reposo, no ZK total
Servidor propio con proceso que descifra	Sí, el servidor puede leerlo	Servidor privado
WebCrypto con LLM externo	Sí, el proveedor recibe el texto	Cifrado local parcial

La documentación debe especificar siempre: dónde se cifra, dónde se descifra, quién puede ver plaintext, qué se envía a proveedores, qué se almacena y cuánto tiempo se conserva.

6. Criptografía

6.1. Datos en reposo

- AES-256-GCM.
- XChaCha20-Poly1305 cuando la librería soporte el algoritmo correctamente.
- Envelope encryption para servidores y backups.

6.2. Derivación de claves

- Argon2id preferido para claves derivadas desde passphrase.
- Salt aleatorio único.
- Parámetros documentados y política de migración si se actualizan.
- Verificación de versión criptográfica.
- PBKDF2 puede mantenerse por compatibilidad web cuando sea necesario, documentado como decisión de compatibilidad.

6.3. Claves por capas

Las claves de integración no deben derivarse de una memoria ni almacenarse junto al contenido.

7. Gestión de identidad

7.1. Contraseñas y sesiones

- Argon2id para contraseñas con longitud mínima razonable.
- Rate limit y mensajes que no revelen existencia de cuentas.
- Cookies HttpOnly, Secure y SameSite.
- Expiración corta para access session + refresh token rotatorio.
- Revocación server-side y detección básica de reutilización.
- MFA posterior: TOTP, WebAuthn/passkeys, códigos de recuperación.

8. Autorización y aislamiento

8.1. Capas de autorización

8.2. Workspace isolation

Cada query debe filtrar por workspace_id antes de:

- Búsqueda textual.
- Búsqueda vectorial.
- Recorrido de grafo.
- Lectura de archivos.
- Selección de contexto para LLM.

Denegación por defecto: si no hay una política explícita, la operación se deniega o queda en espera de confirmación.

9. Seguridad del frontend

- **XSS:** Sanitizar Markdown y HTML, no insertar directamente contenido de LLM, usar CSP, evitar dangerouslySetInnerHTML.
- **Almacenamiento local:** No guardar en localStorage API keys, refresh tokens, passphrases, secretos ni plaintext sensible. Usar IndexedDB cifrado, keychain del sistema, Secure Storage Android, cookies HttpOnly.
- **Archivos:** Validar MIME real, limitar tamaño, no ejecutar archivos subidos, no permitir path traversal, crear nombres internos.
- **CSP estricta:** Mantener actualizada. No permitir unsafe-eval salvo dependencia indispensable justificada.
- **Micrófono:** Solicitar permiso justo antes del uso, mostrar estado de grabación, no grabar en background sin indicación.
- **Clickjacking:** frame-ancestors none, X-Content-Type-Options nosniff, Referrer-Policy restrictiva, enlaces externos con noopener norereferrer.

10. Seguridad del backend

- **Validación:** Todos los endpoints validan tipo, longitud, enumeraciones, fechas, tamaño de archivos, URLs, identificadores y JSON estructurado.
- **Rate limiting:** Por IP, usuario, workspace, agente, proveedor, herramienta, tipo de operación y coste/token. En despliegues múltiples usar Redis u otro almacén compartido.
- **SSRF:** Bloquear IPs privadas por defecto, resolver DNS y verificar destino final, limitar redirects, aplicar timeout, limitar respuesta, usar allowlist configurable.
- **SQL y comandos:** Usar queries parametrizadas/ORM seguro, no ejecutar comandos del sistema desde texto del usuario, no permitir shell a agentes en MVP.

- share.create
- Herramientas con efectos externos irreversibles.

12.2. Aislamiento

- Credenciales por integración.
- Timeouts y límites de respuesta.
- Límite de tool calls.
- Sin acceso a variables de entorno globales.
- Red restringida.
- No ejecutar servidores externos como root.
- Revocación inmediata.

13. Logs, auditoría y telemetría

13.1. Logs operativos permitidos

- Request ID, Operation ID, Job ID.
- Estado, duración, error code.
- Modelo/proveedor, conteos, tamaño agregado.

13.2. No registrar por defecto

- Plaintext, audio, passwords.
- API keys, OAuth tokens, cookies.
- Contenido completo de email.
- Prompt completo si contiene datos privados.

Telemetría opt-in: debe estar desactivada por defecto en modo privado, no incluir contenido, ser agregada, documentar eventos, permitir desactivar y no utilizar identificadores personales innecesarios.

14. Backups y recuperación

- El backup debe cifrarse antes de salir del dispositivo o servidor.
- La clave de backup no debe almacenarse junto al archivo. Puede conservarse en passphrase del usuario, keychain, secret manager o hardware key futura.
- Un backup no se considera válido hasta que pueda restaurarse en un entorno de prueba.

- Ransomware y corrupción: mantener versiones, checksums, backups offline o inmutables cuando sea posible, detectar cambios anómalos, no sobrescribir el único backup.

15. Gestión de incidentes

15.1. Clasificación

Nivel	Descripción
P0	Exposición activa o pérdida masiva.
P1	Vulnerabilidad explotable o servicio crítico afectado.
P2	Impacto limitado o configuración incorrecta.
P3	Mejora o vulnerabilidad de bajo impacto.

15.2. Flujo de respuesta

Detectar
Contener
Evaluar alcance
Reparar

16. Testing de seguridad

16.1. Automatizado en CI (obligatorio desde día uno)

Herramienta	Propósito	Frecuencia
Gitleaks	Detección de secretos en commits	Cada PR
Semgrep	SAST (análisis estático)	Cada PR
npm audit / pip-audit	Vulnerabilidades en dependencias	Cada PR y nightly
Trivy	Scan de contenedores Docker	Cada build de imagen

16.2. Antes de cada release

- DAST con OWASP ZAP para todos los endpoints públicos.
- Dependencias con Snyk / Gype para todo el árbol.
- Permisos: revisión manual de archivos de configuración, Docker, CI.
- Secrets con TruffleHog para historial completo del repo.

16.3. Antes de v1.0

- Pentest básico por un revisor externo.

- Revisión del threat model completo.
- Revisión de todos los scopes de MCP.
- Revisión de backups y restore.
- Revisión de exportación/importación.
- Test de recuperación ante desastre.

17. Criterios de aceptación de seguridad

VNBOT cumple el baseline de seguridad cuando satisface 20 criterios:

01. Las contraseñas no se almacenan en texto.
02. Los tokens se pueden revocar.
03. Un usuario no puede leer otro workspace.
04. Un agente no puede usar una herramienta no autorizada.
05. Las acciones de riesgo requieren confirmación.
06. Las respuestas de LLM no ejecutan código directamente.
07. MCP está sujeto a scopes y timeouts.
08. Los archivos no permiten path traversal.
09. Los logs no contienen secretos.
10. Las claves no viven en localStorage.
11. La aplicación tiene CSP y sanitización.
12. Los backups están cifrados.
13. Existe exportación y eliminación.
14. Se pueden reconstruir índices derivados.
15. Hay healthchecks y alertas.
16. Existe SECURITY.md.
17. La CI escanea secretos y dependencias.
18. Los releases tienen checksums o firma.
19. La política de privacidad distingue local, servidor privado y cloud.
20. Las integraciones oficiales se documentan con sus scopes y límites.

18. Conclusión

La seguridad de VNBOT no debe depender de una única función de cifrado. Debe surgir de la combinación de:

Minimización de datos
+ aislamiento por workspace
+ permisos por agente
+ scopes MCP

VNBOT debe ser extensible en capacidades, pero restrictivo en autoridad.

DOC 09 • MCP Y SKILLS

Tools, scopes, agentes, autonomía, marketplace futuro y contract testing.

1. Propósito

VNBOT debe poder crecer sin que cada nueva capacidad obligue a modificar el núcleo de memoria, recordatorios y usuarios. Para ello utilizará dos mecanismos relacionados, pero diferentes: MCP (protocolo para conectar herramientas, recursos y prompts externos o internos) y Skills (capacidades de comportamiento versionadas que indican cómo realizar una tarea y qué herramientas necesitan).

Los agentes son la combinación de:

Modelo
+ instrucciones
+ skills
+ memoria permitida
+ herramientas permitidas
+ po

La extensibilidad no debe significar autoridad ilimitada. Un agente puede saber cómo usar una herramienta, pero el policy engine decide si tiene permiso para usarla en una operación concreta.

2. Principios

Principio	Descripción
MCP no es autorización	MCP define cómo intercambiar contexto y herramientas. No decide si una acción es segura para VNBOT. La autorización se implementa en el gateway y en el dominio.
Deny by default	Una herramienta nueva aparece deshabilitada hasta que se registra, inspecciona, asigna scope, revisa riesgo y el usuario la activa.
El LLM propone, el policy engine decide	LLM → propone tool call · Schema → valida argumentos · Policy → valida permiso/riesgo · Usuario → confirma cuando aplica · Executor → ejecuta · Audit → registra.
Memoria mínima	Un agente recibe el mínimo contexto necesario para una skill. No se inyecta toda la bóveda por defecto.
Herramientas pequeñas	Las tools deben tener responsabilidades concretas. Es preferible calendar.create_event + calendar.list_events + calendar.delete_event en lugar de system.do_anything.
Reversibilidad	Las acciones deben ser reversibles o requerir confirmación si no lo son.
Versionado	MCP servers, tools, skills, manifests y agentes deben poder versionarse y migrarse.

3. Arquitectura MCP de VNBOT

3.1. MCP interno (13 tools)

3.2. MCP externo

- Graphify (repos de código).
- Calendarios (Google, ICS, CalDAV).
- Email (Gmail, IMAP).
- Filesystem (carpetas locales).
- Web/search.
- Notas (Obsidian, Notion).
- Mensajería oficial (Telegram, WhatsApp Business).
- Herramientas de desarrollo.

4. Registro de servidores MCP

4.1. Flujo de registro

4.2. Registro local (stdio)

El comando debe ejecutarse con un usuario sin privilegios y con directorios limitados.

4.3. Registro remoto (Streamable HTTP)

Las credenciales se almacenan en un secret store. No se guardan en el documento de configuración público.

5. Scopes y permisos

5.1. Scopes iniciales (15 scopes)

graph.read · graph.write
memory.read · memory.write · memory.delete
calendar.read · calendar.write

5.2. Niveles de permiso

- **deny** — denegado.
- **read** — solo lectura.
- **write** — escritura.
- **execute** — ejecución.
- **admin** — administración (no autorización genérica para todo el sistema).

5.3. Matriz de permisos comprensible

La UI debe presentar una matriz comprensible:

Agente: Beacon

Memoria personalLeer ✓Escribir ✓Borrar x
Calendario

6. Clasificación de riesgo de herramientas

Nivel	Ejemplos
Bajo	Buscar memoria, leer calendario, listar tareas, consultar healthcheck, crear lista local, generar resumen.
Medio	Crear memoria, crear recordatorio, crear evento, actualizar preferencia, sincronizar documento.
Alto	Leer emails completos, crear borradores, compartir información, leer directorios personales, enviar notificaciones a terceros, modificar eventos existentes.
Crítico	Enviar emails, borrar datos masivamente, escribir archivos arbitrarios, cambiar permisos, acciones financieras, ejecutar comandos del sistema.

Las acciones críticas no estarán disponibles en el MVP o exigirán confirmación fuerte, reautenticación y límites adicionales.

7. Confirmaciones

7.1. Confirmación simple (riesgo bajo/medio)

Voy a crear este recordatorio:
Título: Revisar presupuesto
Fecha: lunes 20 de julio, 09:00

7.2. Confirmación fuerte (riesgo alto)

- Mostrar tool exacta.
- Mostrar destino y contenido completo.
- Mostrar scopes utilizados y agente.
- Requerir interacción explícita.
- Solicitar reautenticación si es necesario.

7.3. Confirmación caducada

Una propuesta no confirmada después de su TTL debe pasar a EXPIRED. Si el usuario intenta confirmarla, debe recalcularse.

8. Skills

8.1. Estructura de una skill

8.2. Manifest YAML

8.3. Skills iniciales (19 skills)

9. Ciclo de vida de una skill

9.1. Actualización de skills

Si cambia cualquiera de estos campos se requiere revisión del usuario:

- required_tools
- memory_scopes
- risk_level
- confirmation_policy

10. Agentes

10.1. Configuración de un agente

Agent
- name · description · avatar
- model · system instructions
- skills · memory scope

10.2. Agentes iniciales (7 agentes)

Agente	Orientación	Herramientas externas
VNBOT Core	Asistente general de bajo riesgo	Buscar memoria y proponer recordatorios
Archivista	Guardar, organizar y recuperar memorias	No tiene herramientas externas por defecto
Beacon	Recordatorios y tareas	Crear y gestionar recordatorios internos
Navigator	Calendario	Inicia con lectura y propone eventos
Forge	Proyectos e ideas	Crear listas, notas y relaciones
Sentinel	Seguridad, permisos, healthchecks y auditoría	No ejecuta acciones externas
Scout	Investigación con web/MCP	Trata contenido externo como no confiable

11. Niveles de autonomía (0-4)

Nivel	Nombre	Capacidades
0	Responder	El agente solo genera respuestas. No crea datos.
1	Proponer	Puede presentar memorias, recordatorios o tool calls para aprobación.
2	Acciones internas	Puede crear y modificar datos internos según skills permitidas.
3	Integraciones confirmadas	Puede preparar acciones externas y ejecutarlas después de confirmación.
4	Automatización limitada	Puede ejecutar reglas explícitas bajo horario, presupuesto, lista cerrada de herramientas, máximo de llamadas, registro y botón de parada.

Nunca debe existir una opción "sin límites" en el sentido de acceso total al sistema.

12. Memoria accesible por agente

12.1. Scopes de memoria

personal · work · study · family
project:vnbot · sensitive · secret

12.2. Reglas de memoria por agente

- secret no se incluye en prompts externos.
- Un agente solo consulta scopes asignados.
- Los scopes se filtran antes del LLM.
- La UI muestra qué scopes utiliza.
- Un agente no puede cambiar sus propios scopes.

12.3. Context assembly

13. Presupuesto y límites

13.1. Límites por agente

13.2. Exceso de presupuesto

El sistema debe detenerse con estado explícito BUDGET_EXCEEDED. No debe cambiar silenciosamente a un proveedor más costoso.

14. Tool execution — preflight, ejecución y post

14.1. Preflight (antes de ejecutar)

- Tool existe.
- Servidor healthy.
- Schema válido.
- Scope presente.
- Usuario autorizado.
- Presupuesto disponible.
- Confirmación vigente.
- Idempotency key.

14.2. Ejecución

- Timeout.
- Cancelación.
- Captura de status.
- Sanitización de resultado.
- Límite de respuesta.

14.3. Post-execution

- Guardar resumen.
- Actualizar recurso interno si corresponde.
- Invalidar cache.
- Registrar auditoría.
- Actualizar UI.
- Mostrar resultado y warnings.

15. Contract testing para herramientas MCP

Cada herramienta MCP integrada en VNBOT debe tener contract tests que verifiquen su comportamiento sin depender de un servidor MCP real.

Aspecto	Verificación
Input schema	El schema declarado coincide con lo que la tool realmente acepta.
Output schema	El output real coincide con el schema declarado.
Scopes	La tool solo accede a los recursos que sus scopes permiten.
Aislamiento	Un fallo en la tool no propaga al resto del sistema.
Timeout	La tool respeta el timeout configurado.
Idempotencia	Llamar la tool dos veces con los mismos args produce el mismo resultado (cuando aplica).

15.1. Métricas de tools

```
vnbot_mcp_tool_calls_total{tool_name, status}<br/>vnbot_mcp_tool_duration_seconds{tool_name}<br/>vnbot
```

16. Criterios de aceptación

16.1. MCP está correctamente implementado cuando (13 criterios)

01. Se puede registrar un servidor local o remoto.
02. El handshake queda auditado.
03. Las tools descubiertas aparecen deshabilitadas por defecto.
04. El usuario selecciona scopes.
05. El agente solo ve herramientas autorizadas.
06. Las llamadas tienen schemas.
07. Las operaciones de riesgo requieren confirmación.
08. Existen timeouts y límites.
09. El servidor puede revocarse.
10. El fallo de un MCP no rompe VNBOT.
11. Los logs no contienen secretos.
12. Graphify funciona como integración opcional.
13. Las respuestas externas se tratan como datos no confiables.

16.2. Una skill está lista cuando

- Tiene manifest, licencia y input/output schema.
- Declara herramientas, memoria y riesgo.
- Tiene instrucciones claras, tests y simulación.
- Tiene manejo de error.
- Puede desactivarse.
- No puede ampliar permisos sola.

16.3. Un agente está listo cuando

- Tiene identidad, propósito y mascota/estado.
- Tiene modelo configurable, skills definidas y scopes de memoria.
- Tiene tools permitidas y autonomía explícita.
- Tiene presupuesto, simulación y auditoría.
- Puede pausarse y revocarse.

17. Conclusión

MCP y las skills permiten que VNBOT crezca desde una aplicación de recordatorios hasta una plataforma de agentes personalizables. Pero esa expansión solo es sostenible si cada capacidad tiene límites claros.

La arquitectura de VNBOT queda definida así:

Skill describe cómo trabajar.
Agente define quién trabaja.
MCP conecta qué puede usar.
Pol

Una herramienta puede estar conectada sin estar autorizada; un agente puede conocer una skill sin tener permiso para ejecutarla.